LOW-LEVEL DESIGN DOCUMENT

RAG-Based Customer Support Assistant

with LangGraph and Human-in-the-Loop

Prepared by: Kartik Patade

1. Module-Level Design

1.1 Document Processing Module

Responsibility: Load the PDF from disk and extract text content.

Implementation: Uses PyPDFLoader. Returns a list of Document objects, each representing one page.

```
loader = PyPDFLoader('knowledge_base.pdf')
documents = loader.load()
```

1.2 Chunking Module

Responsibility: Split the raw document text into smaller, overlapping pieces.

Implementation: Uses RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=50.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
```

Why these values: 500 characters keeps chunks concise while containing enough context. 50 character overlap prevents information loss at boundaries.

1.3 Embedding Module

Responsibility: Convert text chunks and user queries into vector representations.

Implementation: Uses HuggingFaceEmbeddings with the all-MiniLM-L6-v2 model. This runs locally with no API cost.

```
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
```

1.4 Vector Storage Module

Responsibility: Store embeddings persistently and support similarity search.

Implementation: ChromaDB with a persistent directory (./chroma_store). During ingestion, from_documents() creates the collection. During query time, as_retriever() provides the search interface.

```
vectorstore = Chroma.from_documents(chunks, embeddings,
persist_directory='./chroma_store')
vectorstore.persist()
```

1.5 Retrieval Module

Responsibility: Find the most relevant chunks for a given query.

Implementation: The Chroma retriever uses cosine similarity. Returns top k=3 chunks.

```
retriever = vectorstore.as_retriever(search_kwargs={'k': 3})
retrieved_docs = retriever.invoke(query)
```

1.6 Query Processing Module

Responsibility: Format the retrieved chunks and user query into a prompt, then call the LLM.

Implementation: Constructs a prompt string combining context chunks and the query. Calls the Gemini API. Returns the generated text.

```
context = '\n'.join([doc.page_content for doc in retrieved_docs])
prompt = f'Context:\n{context}\n\nQuestion: {query}\nAnswer:'
```

1.7 Graph Execution Module (LangGraph)

Responsibility: Orchestrate the overall query workflow using a stateful graph.

Implementation: Defines a StateGraph with a TypedDict state schema. Adds nodes for retrieval/generation and routing. Sets conditional edges based on the response content.

1.8 HITL Module

Responsibility: Handle escalation when the system cannot produce a confident answer.

Implementation: Checks the response for uncertainty signals. Logs the query and simulates escalation to a human agent.

2. Data Structures

2.1 Document Representation

Each page from the PDF is stored as a LangChain Document object:

```
Document(
  page_content: str,   # raw text from that page
  metadata: {
    source: str,       # filename
    page: int          # page number
  }
)
```

2.2 Chunk Format

After splitting, each chunk is still a Document object but with shorter page_content and the same metadata fields.

2.3 Embedding Structure

Each chunk is represented as a 384-dimensional float vector (output of MiniLM-L6-v2). These are stored in ChromaDB along with the original text and metadata for retrieval.

2.4 Query-Response Schema

```
{
  query: str,
  retrieved_chunks: List[str],
  llm_response: str,
  escalated: bool
}
```

2.5 LangGraph State Object

```
class GraphState(TypedDict):
  query: str
  context: str
  response: str
  escalate: bool
```

3. Workflow Design (LangGraph)

3.1 Nodes

retrieve_and_generate node

Input: GraphState with query set.

Process: Retrieves top 3 chunks from ChromaDB. Builds prompt. Calls LLM. Stores response in state.

Output: Updated GraphState with context and response fields populated.

route_response node

Input: GraphState with response set.

Process: Checks for low confidence signals in the response. Sets escalate flag accordingly.

Output: Updated GraphState with escalate field set.

output_node

Input: Final GraphState.

Process: If escalate is False, prints the response. If escalate is True, triggers HITL escalation function.

3.2 Edges

START -> retrieve_and_generate

retrieve_and_generate -> route_response (always)

route_response -> output_node (conditional: based on escalate flag)

output_node -> END

3.3 State Flow

State starts with only query filled. After retrieve_and_generate, context and response are added. After route_response, escalate is set. The output node reads all fields and acts accordingly.

4. Conditional Routing Logic

4.1 Answer Generation Criteria

The system generates and delivers an answer directly when: the retriever finds at least one relevant chunk, and the LLM response does not contain any of the uncertainty phrases listed below.

4.2 Escalation Criteria

The system escalates when any of these conditions are true:

- Retrieved chunks list is empty (no relevant content found)
- LLM response contains phrases like: 'I don't know', 'I cannot find', 'not sure', 'no information available', 'cannot answer'
- Query contains words like 'legal', 'billing dispute', 'refund claim', or 'complaint' (complex query flag)

These checks are simple string operations. No ML classifier is used for routing.

```
UNCERTAINTY_PHRASES = ['i don\'t know', 'cannot find', 'not sure', 'no
information']
def should_escalate(response, retrieved_docs, query):
  if not retrieved_docs:
    return True
  for phrase in UNCERTAINTY_PHRASES:
    if phrase in response.lower():
      return True
  return False
```

5. HITL Design

5.1 When Escalation is Triggered

Escalation happens inside the route_response node when should_escalate() returns True. This updates the GraphState with escalate=True.

5.2 What Happens After Escalation

The output_node detects escalate=True and calls the handle_hitl() function. Currently this function prints a formatted message showing the original query and the attempted (but uncertain) response. It also logs the entry to a local escalation_log.txt file with a timestamp.

```
def handle_hitl(state):
  print('[ESCALATED] Query flagged for human review.')
  print(f'Query: {state["query"]}')
  with open('escalation_log.txt', 'a') as f:
    f.write(f'{datetime.now()} | {state["query"]}\n')
```

5.3 How Human Response is Integrated

In this implementation, human integration is simulated. After a query is logged as escalated, the system waits for manual input from the console. A real production version would push the query to a ticketing system (like Zendesk) and pull the human response back in once resolved.

6. API / Interface Design

6.1 Input Format

The system accepts input as a plain text string entered at the command line. The main loop reads from stdin.

```
query = input('Enter your question: ')
```

6.2 Output Format

If answered: The response is printed to stdout with a label.

```
Answer: <LLM response text>
```

If escalated: The escalation message is shown instead.

```
[ESCALATED] This query has been flagged for human review.
```

6.3 Interaction Flow

- User runs the app from terminal
- App checks if ChromaDB already has data; if not, runs ingestion first
- User enters a query
- Graph executes, response is displayed
- Loop repeats until user types 'exit'

7. Error Handling

7.1 Missing Data

If the PDF file path is incorrect or the file does not exist, the app catches the FileNotFoundError and prints a clear message asking the user to check the file path. Ingestion is not attempted.

7.2 No Relevant Chunks Found

If the retriever returns an empty list (which can happen if the query is too different from the document content), the should_escalate() function catches this via the empty list check and routes to HITL.

7.3 LLM Failure

If the Gemini API call fails (timeout, quota exceeded, etc.), the code wraps it in a try-except block. On failure, the state response is set to a fallback string that contains an uncertainty phrase, which automatically triggers escalation.

```
try:
  response = llm.invoke(prompt)
except Exception as e:
  response = 'I cannot find an answer at this time.'
```